

Abstraction Patterns for Cross Platform Development

Al-Afiq Yeong

2024

Table of Content

1. Introduction
2. Preprocessor (#if, #else, #elif)
3. Runtime Polymorphism
4. PIMPL
5. Build Systems
6. ??????
7. C++ 20 Modules
8. Conclusion

Introduction

Definition of “Abstraction”

English	Software Engineering
Originated from the word “Abstract” - Disassociated from any specific instance.	The process of removing or generalizing physical, spatial, or temporal details or attributes in the study of objects or systems to focus attention on details of greater importance; it is similar in nature to the process of generalization.

TLDR - Making code generic.

“Cross Platform” can be:

- Multiple operating systems.
- Different APIs or Libraries
- Hardware
- Software

Challenges:

- Generally difficult. Require knowledge of all platforms.
- Require a lot of planning.
- Takes a while to strike a balance.

Why?

- Unified interface that exposes a single source of truth to users.
- Easier to maintain over the long term by preventing code divergence.
- Saves development cost (long term).
- Enables new opportunities. Easily extendable if required to support other platforms.

Context

1. We are making a new renderer that supports multiple graphics API backend.
2. We need to support at least 2 graphics API on launch; DirectX12 and Vulkan. Possibly more in the future.

For each pattern:

1. Create a graphics device that holds an instance of the API of choice.
2. Add a few functions that the device can execute.
3. Discuss its structure, pros, and cons.

Disclaimer

Will only touch the surface on how each feature work internally. Not in detail.

For e.g,

1. In depth explanation of C++ 20 modules.
2. Build systems.

Preprocessor

(#if, #elif, #else)

- Simplest way to abstract cross platform code.
- The first method any C++ engineer would think of when attempting to write cross platform code.
- Works really well on small pieces of code (1-3 lines).

```

// device.cpp
#ifdef USE_VULKAN
#include <vulkan.h>
#elif USE_DX12
#include <d3d12.h>
#endif

using device_ptr = void*;

auto create_device() -> device_ptr
{
#ifdef USE_VULKAN
    // 40 line Vulkan setup code here ...

    VkDeviceCreateInfo info = { ... };

    VkDevice vkDeviceHandle = VK_NULL_HANDLE;

    if (vkCreateDevice(gpu, &info, nullptr, &vkDeviceHandle) == VK_SUCCESS)
    {
        return vkDeviceHandle;
    }
#elif USE_DX12

    DXGI_SWAP_CHAIN_DESC1 sd = { ... }; // DX12 setup code.

    ID3D12Device* dx12DeviceHandle = nullptr

    if (D3D12CreateDevice(nullptr, featureLevel, IID_PPV_ARGS(&dx12DeviceHandle)) == S_OK)
    {
        return dx12DeviceHandle;
    }
#endif

    return nullptr;
}

```

Problems:

- Function body can become really long and grows exponentially the more platforms you support.
- Impossible to read.
- Merging code becomes extremely difficult (Harder than it already is).
- Ugly code structure, macro pollution.
- Requires preprocessor to be defined / exposed across all projects.

Don't do this!

Runtime Polymorphism

- Using a derived class to define implementation from an abstract / virtual base class through the use of encapsulation and overriding virtual methods.
- Quick refresher on runtime polymorphism.
 - When a method in a class has the virtual specifier, compiler generates a vtable for the class.
 - Lookup table of functions to resolve function calls.
 - vtable exist for each class type.
 - Compiler also generates a hidden virtual pointer that points to the virtual table of that class.

```
// device.hpp
```

```
class IBuffer { ... };
```

```
class IImage { ... };
```

```
class ISwapchain { ... };
```

```
class IDevice
```

```
{
```

```
public:
```

```
    virtual ~IDevice() = 0;
```

```
    virtual auto allocate_buffer() -> std::shared_ptr<IBuffer> = 0;
```

```
    virtual auto create_image() -> std::shared_ptr<IImage> = 0;
```

```
    virtual auto create_swapchain() -> std::shared_ptr<ISwapchain> = 0;
```

```
}
```

```
auto make_vulkan_device() -> std::unique_ptr<IDevice>;
```

```
auto make_dx12_device() -> std::unique_ptr<IDevice>;
```

Implementation can be declared in header files too but will leak out platform details.

```
// vulkan.impl.cpp
#include <vulkan.h>

class VulkanBuffer : public IBuffer { ... }
class VulkanImage : public IImage { ... }
class VulkanSwapchain : public ISwapchain { ... }

class VulkanDevice final : public IDevice
{
private:
    VkDevice m_device;
public:
    VulkanDevice();
    ~VulkanDevice();

    auto allocate_buffer() -> std::shared_ptr<IBuffer> override;
    auto create_image() -> std::shared_ptr<IImage> override;
    auto create_swapchain() -> std::shared_ptr<ISwapchain> override;
}

auto make_vulkan_device() -> std::unique_ptr<IDevice>
{
    return std::make_unique<VulkanDevice>();
}
```

```
//dx12.impl.cpp
#include <d3d12.h>

class DX12Buffer : public IBuffer { ... }
class DX12Image : public IImage { ... }
class DX12Swapchain : public ISwapchain { ... }

class DX12Device final : public IDevice
{
private:
    ID3D12Device* m_device;
public:
    DX12Device();
    ~DX12Device();

    auto allocate_buffer() -> std::shared_ptr<IBuffer> override;
    auto create_image() -> std::shared_ptr<IImage> override;
    auto create_swapchain() -> std::shared_ptr<ISwapchain> override;
}

auto make_dx12_device() -> std::unique_ptr<IDevice>
{
    return std::make_unique<DX12Device>();
}
```

Pros	Cons
● Language feature. Easy to get started. ● Allows consistent and non divergent paths in the codebase. ● Same interface regardless of the platform. ○ Object can be passed around as a pointer to the base class. ○ vfptr calls the right method (as long as it's overridden correctly). ● Merge conflicts, not so painful to read structure wise.	● Performance cost. ○ Vtable lookup. ○ Missed out inlining opportunities. ○ Devirtualization <i>*might*</i> not happen. ● Final executable contains code from other platform implementations. ● Platform specific declarations and signatures can leak. ● Method duplication.

It's 2024, let's try something ~~better~~ **different!** 😏

PIMPL

PIMPL, Pointer to Implementation.

Usually involves some form of type erasure by:

- Using a void*
- Using a pointer to a forward declared implementation object.
 - `std::unique_ptr` can be used to properly manage lifetime.

For both approach:

1. An object that contains implementation details is defined in a source file.
2. Point the pointer to an instance of the implementation.

```
// device.hpp
```

```
class Device
```

```
{
```

```
public:
```

```
    Device();
```

```
    ~Device();
```

```
    auto allocate_buffer() -> resource_ptr<Buffer>;
```

```
    auto create_image() -> resource_ptr<Image>;
```

```
    auto create_swapchain() -> resource_ptr<Swapchain>;
```

```
private:
```

```
    void* m_impl;
```

```
}
```

```
// device.hpp

// Forward declare.
struct DeviceImpl;

class Device
{
public:
    Device();
    ~Device();

    auto allocate_buffer() -> resource_ptr<Buffer>;
    auto create_image() -> resource_ptr<Image>;
    auto create_swapchain() -> resource_ptr<Swapchain>;

private:

    using device_impl_ptr = std::unique_ptr<DeviceImpl>;

    device_impl_ptr m_impl;
}
```

```
// device.hpp

// Forward declare.
struct DeviceImpl;

class Device
{
public:
    Device();
    ~Device();

    auto allocate_buffer() -> resource_ptr<Buffer>;
    auto create_image() -> resource_ptr<Image>;
    auto create_swapchain() -> resource_ptr<Swapchain>;

private:
    friend auto create_device(CreateDeviceInfo const& info) -> std::optional<std::unique_ptr<Device>>;

    Device(std::unique_ptr<DeviceImpl> implPtr);

    using device_impl_ptr = std::unique_ptr<DeviceImpl>;

    device_impl_ptr m_impl;
}
```

```

// device.vulkan.impl.cpp
#include <vulkan.h>

struct DeviceImpl
{
    VkInstance instance;
    VkPhysicalDevice gpu;
    VkDevice device;
    // all other platform specific objects here ...

    auto allocate_buffer() -> resource_ptr<Buffer>
    {
        vkCreateBuffer(...);
    }

    auto create_image() -> resource_ptr<Image>
    {
        vkCreateImage(...);
    }

    auto create_swapchain() -> resource_ptr<Swapchain>
    {
        vkCreateSwapchainKHR(...);
    }
}

```

```

// device.dx12.impl.cpp
#include <d3d12.h>

struct DeviceImpl
{
    ID3D12Device device;
    IDXGIFactory4 factory;

    auto allocate_buffer() -> resource_ptr<Buffer>
    {
        device->CreateBuffer(...);
    }

    auto create_image() -> resource_ptr<Image>
    {
        device->CreateCommittedResource(...);
    }

    auto create_swapchain() -> resource_ptr<Swapchain>
    {
        factory->CreateSwapchainForHwnd(...);
    }
}

```

```
// device.cpp
```

```
Device::Device(std::unique_ptr<DeviceImpl> implPtr) :
```

```
    m_impl{ std::move(implPtr) }
```

```
{}
```

```
auto Device::allocate_buffer() -> resource_ptr<Buffer>
```

```
{
```

```
    return m_impl->allocate_buffer();
```

```
}
```

```
auto Device::create_image() -> resource_ptr<Image>
```

```
{
```

```
    return m_impl->create_image();
```

```
}
```

```
auto Device::create_swapchain() -> resource_ptr<Swapchain>
```

```
{
```

```
    return m_impl->create_swapchain();
```

```
}
```

```
// device.cpp
```

```
auto create_device(DeviceCreateInfo const& info) -> std::optional<std::unique_ptr<Device>>
{
    auto pImpl = std::make_unique<DeviceImpl>();

    if (pImpl->init(info))
    {
        return std::make_optional(std::make_unique<Device>(std::move(pImpl)));
    }

    return std::nullopt;
}
```

Pros	Cons
● Implementation is self contained. ○ Diff-ing / merging code not so painful. ● Persistent interface regardless of platform. ● No compilation dependencies. ○ Platform implementation is encapsulated. ● Consistent non divergent code paths.	● Method duplication. ● Performance cost. ○ Pointer indirection at every function call.

This doesn't work!

We've broken ODR (One Definition Rule).

- *Each declaration can only have **ONE** definition.*

Build System

Programming language is not just its syntax and rules.

Development tools and environment is an integral part of a programming language.

Build system provides platform-independent methods to configure and generate builds for a target platform.

Natural process for C++ engineers to get involved with project configuration using build systems in due time.

What do they do?

- Generate project / solution files.
- **Define new presets.**
- Alter project properties for different configurations.
 - **Include paths.**
 - Compiler flags.
 - Warning levels, Optimizations.
- Project structure.
 - File organization.
 - **Including / excluding source files per configured preset.**

What “problems” are left unsolved?

- ODR violation.
- Multiple platform implementations in final executable.

Fix is simple, just have one implementation per configuration.

```
// CMakePresets.json
```

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "defaults",
      "hidden": true,
      "generator": "Ninja"
    },
    {
      "name": "windows-base",
      "hidden": true,
      "inherits": "defaults"
    },
    {
      "name": "windows-vulkan",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_VULKAN": "ON"
      }
    },
    {
      "name": "windows-dx12",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_DX12": "ON"
      }
    }
  ]
}
```

```
// CMakeLists.txt
```

```
option(USE_VULKAN "Use Vulkan as Graphics API" OFF)
option(USE_DX12 "Use DX12 as Graphics API" OFF)
```

```
add_library(rhi STATIC "device.h")
```

```
if(USE_VULKAN)
  target_sources(
    rhi
    PRIVATE
      "device.vulkan.impl.cpp"
  )
endif()
```

```
if(USE_DX12)
  target_sources(
    rhi
    PRIVATE
      "device.dx12.impl.cpp"
  )
endif()
```

Now that we've unlocked the power of a build system, can we implement a pattern that has the following qualities:

- No pointer indirection to reach implementation
- No vtable lookup

Yes we can!

?????

CRTP - Curiously Recurring Template Pattern

- An object is derived from a templated base class that has itself as the template parameter.

```
template <class T>
class Base
{
    // methods within Base can use template to access members of Derived
};

class Derived : public Base<Derived>
{
    // ...
};
```

```
// device.hpp
template <class DeviceImpl>
class Device
{
public:
    auto allocate_buffer() -> resource_ptr<Buffer>
    {
        auto deviceImpl = static_cast<DeviceImpl*>(this);

        return deviceImpl->allocate_buffer();
    }
    auto create_image() -> resource_ptr<Image>;
    auto create_swapchain() -> resource_ptr<Swapchain>;
}
```

This is the interface we want to expose.

Methods are still being duplicated!

```
// device.vulkan.impl.cpp
class VulkanDevice : public Device<VulkanDevice>
{
}
```

Name of the platform is part of the interface!

```
// device.dx12.impl.cpp
class DX12Device : public Device<DX12Device>
{
}
```

Recall, we have build systems now (Yay?)

- Define new build presets

For each preset:

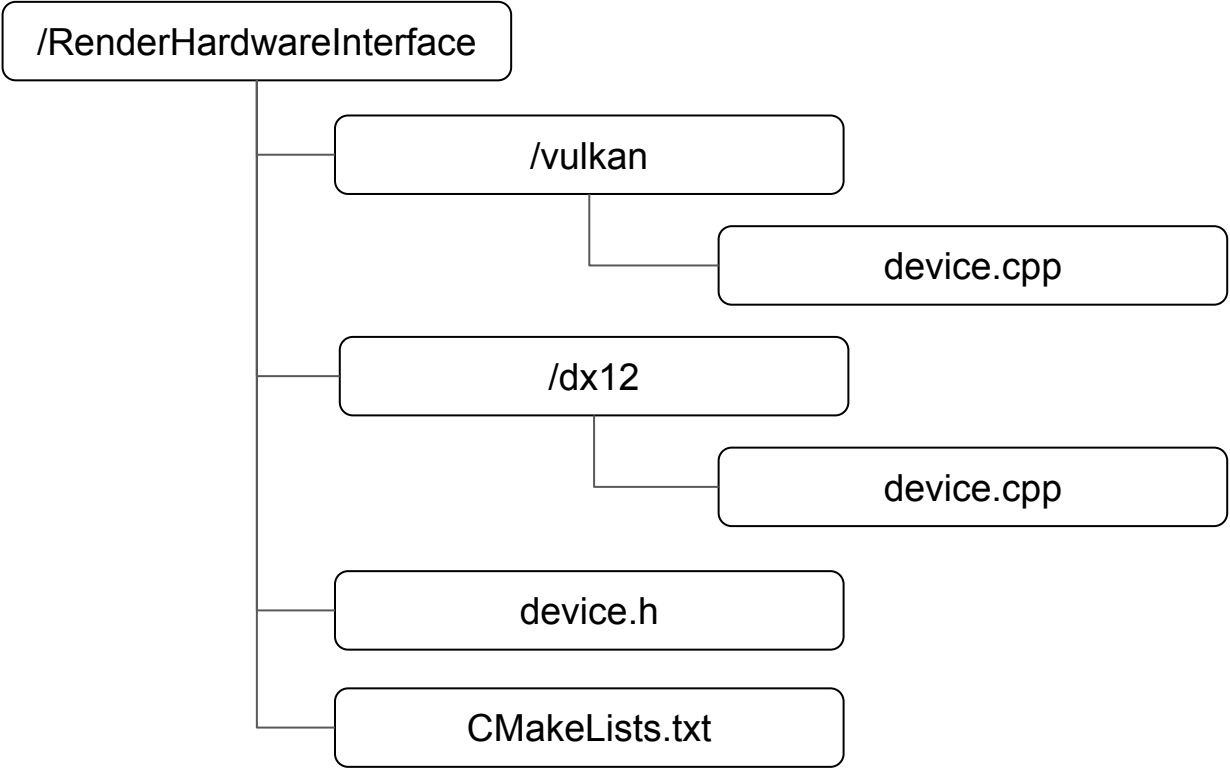
- Have different include paths
- Include and exclude files

When configured correctly, guarantees only one platform per preset.

Interface (base class) does not need to be templated.

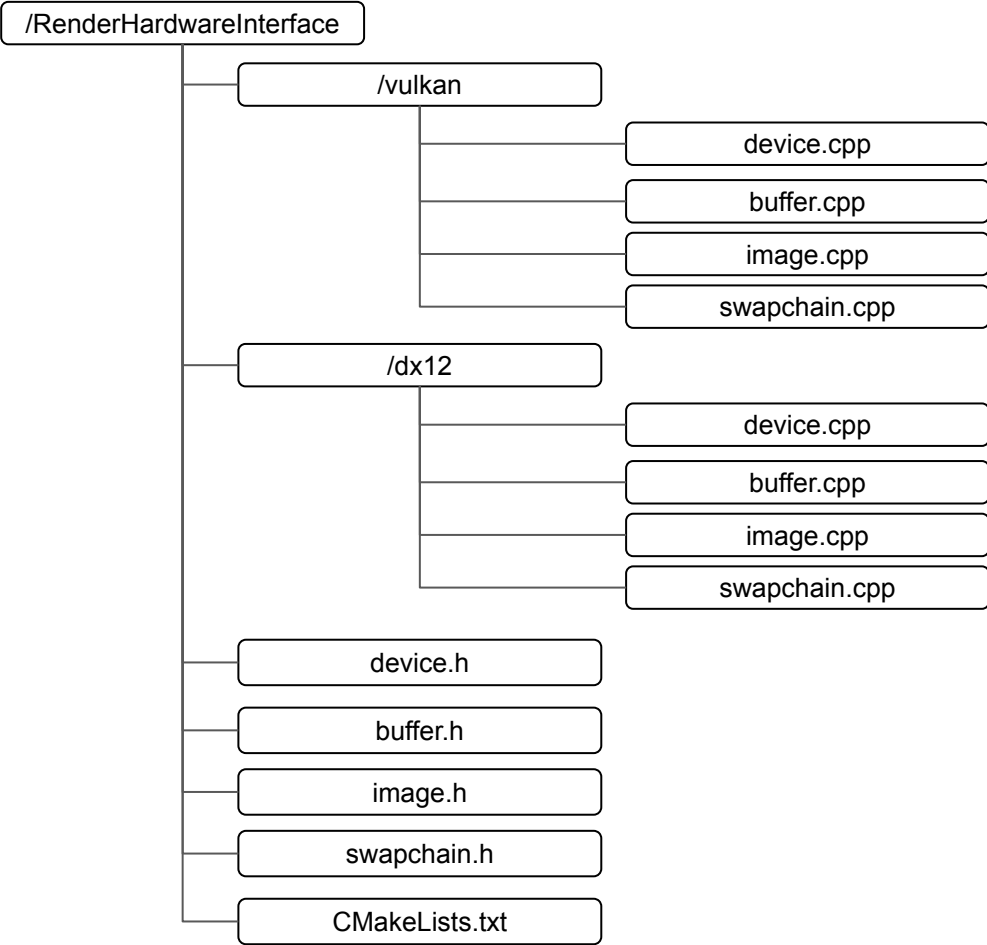
Curiously Recurring ~~Template~~ Pattern

```
// CMakePresets.json
{
  "version": 3,
  "configurePresets": [
    {
      "name": "defaults",
      "hidden": true,
      "generator": "Ninja"
    },
    {
      "name": "windows-base",
      "hidden": true,
      "inherits": "defaults"
    },
    {
      "name": "windows-vulkan",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_VULKAN": "ON"
      }
    },
    {
      "name": "windows-dx12",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_DX12": "ON"
      }
    }
  ]
}
```



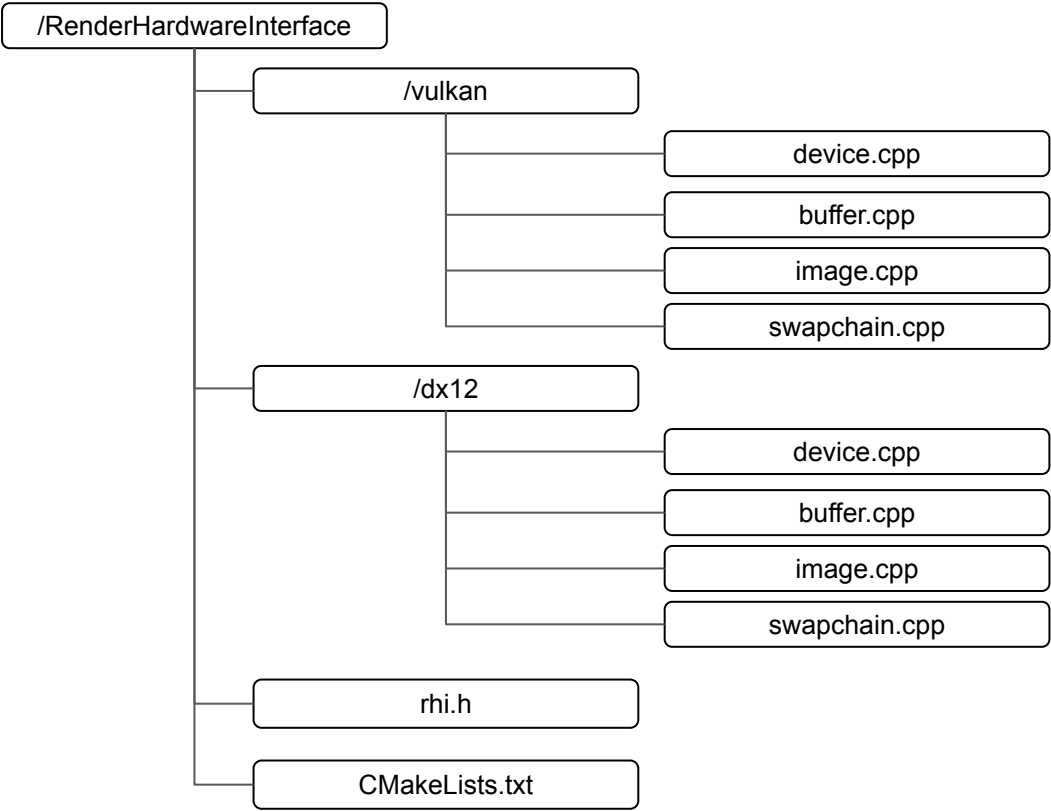
// CMakePresets.json

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "defaults",
      "hidden": true,
      "generator": "Ninja"
    },
    {
      "name": "windows-base",
      "hidden": true,
      "inherits": "defaults"
    },
    {
      "name": "windows-vulkan",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_VULKAN": "ON"
      }
    },
    {
      "name": "windows-dx12",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_DX12": "ON"
      }
    }
  ]
}
```



// CMakePresets.json

```
{
  "version": 3,
  "configurePresets": [
    {
      "name": "defaults",
      "hidden": true,
      "generator": "Ninja"
    },
    {
      "name": "windows-base",
      "hidden": true,
      "inherits": "defaults"
    },
    {
      "name": "windows-vulkan",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_VULKAN": "ON"
      }
    },
    {
      "name": "windows-dx12",
      "inherits": "windows-base",
      "cacheVariables": {
        "USE_DX12": "ON"
      }
    }
  ]
}
```



```
// CMakeLists.txt
```

```
option(USE_VULKAN "Use Vulkan as Graphics API" OFF)
```

```
option(USE_DX12 "Use DX12 as Graphics API" OFF)
```

```
add_library(rhi STATIC "rhi.h")
```

```
if (USE_VULKAN)
```

```
    target_sources(
```

```
        rhi
```

```
        PRIVATE
```

```
            "vulkan/device.cpp"
```

```
            "vulkan/buffer.cpp"
```

```
            "vulkan/image.cpp"
```

```
            "vulkan/swapchain.cpp"
```

```
    )
```

```
endif()
```

```
if(USE_DX12)
```

```
    target_sources(
```

```
        rhi
```

```
        PRIVATE
```

```
            "dx12/device.cpp"
```

```
            "dx12/buffer.cpp"
```

```
            "dx12/image.cpp"
```

```
            "dx12/swapchain.cpp"
```

```
    )
```

```
endif()
```

```

// rhi.h

class Device
{
public:
    Device() = default;
    ~Device() = default;

    auto allocate_buffer() -> resource_ptr<Buffer>;
    auto create_image() -> resource_ptr<Image>;
    auto create_swapchain() -> resource_ptr<Swapchain>;

    auto device_name() -> std::string_view;
    auto driver_version() -> uint32;

    // Other platform agnostic functions.

    static auto from(DeviceCreateInfo const&) -> std::optional<std::unique_ptr<Device>>;
    static auto destroy(std::unique_ptr<Device>&) -> void;

private:
    DeviceCreateInfo m_info      = {};
    std::string m_deviceName     = {};
    uint32 m_driverVersion      = {};
}

```

Generic methods

Platform agnostic
information

```
// vk/device.cpp
```

```
#include "rhi.h"
```

```
class VulkanDevice : public Device
```

```
{  
public:  
    VkInstance instance;  
    VkPhysicalDevice physicalDevice;  
    VkDevice gpu;  
    // all other platform member objects here.  
};
```

```
auto Device::allocate_buffer() -> resource_ptr<Buffer>  
{  
    auto self = static_cast<VulkanDevice*>(this);  
    vkCreateBuffer(self->gpu, ...);  
};
```

```
// dx12/device.cpp
```

```
#include "rhi.h"
```

```
class DX12Device : public Device
```

```
{  
public:  
    ID3D12Device device;  
    IDXGIFactory4 factory;  
};  
  
auto Device::allocate_buffer() -> resource_ptr<Buffer>  
{  
    auto& self = *(static_cast<DX12Device*>(this));  
    self.device->CreateBuffer(...);  
};
```

// vk/vkrhi.h

#include <vulkan.h>

#include "rhi.h"

class VulkanBuffer : public Buffer

{

public:

VkBuffer handle;

VmaAllocation allocation;

VmaAllocationInfo allocationInfo;

};

// + other platform specific implementation i.e Image,
Swapchain, Fence, Pipeline

class VulkanDevice : public Device { ... };

// dx12/dx12rhi.h

#include <d3d12.h>

#include "rhi.h"

class DX12Buffer : public Buffer

{

public:

ID3D12Resource* handle;

D3D12MA::Allocation* allocation;

};

// + other platform specific implementation i.e Image,
Swapchain, Fence, Pipeline

class DX12Device : public Device { ... };

Include these header files
into each cpp file instead

This is all too hacky for me, is there a more standard solution?

C++ 20 Modules

Introduces 2 new keywords, **export** & **import**.

- **Export** - Attached to a declaration and causes that declaration to be *visible* to module importers.
- **Import** - Import a module unit / module partition / header unit.

New properties to be aware of:

- **Visibility** - Candidate for name lookup.
- **Reachability** - Semantics properties become available.

Andreas Weis - [C++ Modules: Getting Started Today](#), CppCon 2023

```
// rhi.ixx/cxx/cppm
```

```
module;
```

```
#include <vulkan.h>
```

```
export module rhi;
```

```
export class Device
```

```
{
```

```
public:
```

```
    auto allocate_buffer() -> resource_ptr<Buffer>;
```

```
    auto create_image() -> resource_ptr<Image>;
```

```
    auto create_swapchain() -> resource_ptr<Swapchain>;
```

```
private:
```

```
    VkInstance instance;
```

```
    VkPhysicalDevice gpu;
```

```
    VkDevice device;
```

```
};
```

```
export class Buffer { ... };
```

```
export class Image { ... };
```

```
export class Swapchain { ... }
```

```
module;
```

```
#include <d3d12.h>
```

```
export module rhi;
```

```
export class Device
```

```
{
```

```
public:
```

```
    auto allocate_buffer() -> resource_ptr<Buffer>;
```

```
    auto create_image() -> resource_ptr<Image>;
```

```
    auto create_swapchain() -> resource_ptr<Swapchain>;
```

```
private:
```

```
    ID3D12Device device;
```

```
    IDXGIFactory4 factory;
```

```
};
```

```
export class Buffer { ... };
```

```
export class Image { ... };
```

```
export class Swapchain { ... }
```

```
// rhi.h
```

```
class IDevice
```

```
{
```

```
public:
```

```
    virtual auto allocate_buffer() -> resource_ptr<Buffer> = 0;
```

```
    virtual auto create_image() -> resource_ptr<Image> = 0;
```

```
    virtual auto create_swapchain() -> resource_ptr<Swapchain> = 0;
```

```
}
```

```
// abstract classes for each platform object.
```

```
class IBuffer { ... };
```

```
class IImage { ... };
```

```
class ISwapchain { ... };
```

Guarantees consistent
interface



```
// rhi.ixx/cxx/cppm
```

```
module;
```

```
// #include <vulkan.h> or #include <d3d12.h>
```

```
#include "rhi.h"
```

```
export module rhi;
```

```
export class Device final : IDevice { ... };
```

```
export class Buffer final : IBuffer { ... };
```

```
export class Image final : IImage { ... };
```

```
export class Swapchain final : ISwapchain { ... };
```

```
export class Device
```

```
{
```

```
private:
```

```
PlatformVariable0 m_foo;
```

```
PlatformVariable1 m_bar;
```

```
PlatformVariable2 m_baz;
```

```
}
```

How to share access?

```
// Provided by VK_VERSION_1_0
void vkCmdBindVertexBuffers(
    VkCommandBuffer          commandBuffer,
    uint32_t                  firstBinding,
    uint32_t                  bindingCount,
    const VkBuffer*           pBuffers,
    const VkDeviceSize*       pOffsets);
```

C++

```
void IASetVertexBuffers(
    [in]          UINT          StartSlot,
    [in]          UINT          NumViews,
    [in, optional] const D3D12_VERTEX_BUFFER_VIEW *pViews
);
```

E.g, Command [Buffer | List] recording, binding a vertex buffer.

What should we do?

- Friend class / function?
- Make them public?
- Getters?
- Disguise them as **void*** ala **std::thread::native_handle()**?



Pssst ... Curiously Recurring ~~Template~~ Pattern



Should really be
rhi.cpp

```
// rhi.ixx/cxx/cppm
```

```
class Device { ... };
```

```
class Buffer
```

```
{
```

```
public:
```

```
    auto info() const -> BufferInfo const&;
```

```
    static auto from(BufferInfo const&) -> resource_ptr<Buffer>;
```

```
    static auto destroy(resource_ptr<Buffer>) -> void;
```

```
private:
```

```
    BufferInfo m_info = {};
```

```
}
```

```
class Image { ... };
```

```
class Swapchain{ ... };
```

```
// platform/buffer.cpp
```

```
module rhi;
```

```
class PlatformBuffer : public Buffer
```

```
{
```

```
public:
```

```
    PlatformVariable0 var0;
```

```
    PlatformVariable1 var1;
```

```
    PlatformVariable2 var2;
```

```
}
```

```
auto Buffer::from(BufferInfo const& info) -> resource_ptr<Buffer>
```

```
{
```

```
    auto platformBuffer = new PlatformBuffer();
```

```
    CreateBuffer(platformBuffer->var0, ...);
```

```
    return resource_ptr{ platformBuffer };
```

```
}
```

Not a silver bullet!

Conclusion

Creating an abstraction for cross platform code is hard.

- Every pattern has unique problems.
- CRP solves most of them.
- Modules are great but can be tricky when dependencies exist.
- Usage of a build system is a must.

If you are feeling bold, Curiously Recurring ~~Template~~ Pattern.

If you are feeling conservative, Runtime Polymorphism + Build System

If you like the latest and greatest, C++ 20 Modules

1. Preprocessor (#if, #else, #elif)
2. Runtime Polymorphism
3. PIMPL
4. Build Systems
5. Curiously Recurring Template Pattern
6. C++ 20 Modules

Thank You!

afiqyeong@gmail.com

<https://ygsm.dev>

<https://www.linkedin.com/in/afiqyeong/>